

Direct Addressing — Exercises 11.1

Exercise 11.1-1

Suppose that a dynamic set S is represented by a direct-address table T of length m . Describe a procedure that finds the maximum element of S . What is the worst-case performance?

Solution

Since each key k corresponds directly to slot $T[k]$:

$$T[k] = \begin{cases} \text{pointer to element} & \text{if } k \in S \\ NIL & \text{otherwise} \end{cases}$$

To find the maximum element, scan from index $m - 1$ down to 0 and return the first non- NIL entry.

Algorithm 1 MAXIMUM(T, m)

```
for  $i \leftarrow m - 1$  downto 0 do
  if  $T[i] \neq NIL$  then
    return  $i$ 
  end if
end for
return  $NIL$ 
```

Worst-case (when S is empty or maximum is near index 0):

$$O(m)$$

Exercise 11.1-2

Describe how to use a bit vector of length m to represent a dynamic set of distinct elements with no satellite data. Dictionary operations should run in $O(1)$ time.

Solution

Use a bit vector $V[0 \dots m - 1]$:

$$V[k] = \begin{cases} 1 & \text{if } k \in S \\ 0 & \text{otherwise} \end{cases}$$

Example (set $\{2, 4, 6, 10, 16\}$ in length 20):

00101010001000001000

Algorithm 2 BITMAP-SEARCH(V, k)

```
if  $V[k] = 1$  then
    return  $k$ 
else
    return NIL
end if
```

Algorithm 3 BITMAP-INSERT(V, k)

```
 $V[k] \leftarrow 1$ 
```

Algorithm 4 BITMAP-DELETE(V, k)

```
 $V[k] \leftarrow 0$ 
```

Each operation runs in:

$$\boxed{O(1)}$$

Exercise 11.1-3

Suggest how to implement a direct-address table in which keys need not be distinct and elements can have satellite data. All operations should run in $O(1)$ time.

Solution

Use a direct-address table where each slot points to a doubly linked list.

$$T[k] \rightarrow \text{list of elements with key } k$$

Operations:

INSERT: Append element to head of list in $O(1)$ time.

DELETE: Given pointer to element, remove from doubly linked list in $O(1)$ time.

SEARCH: Return head of list in $O(1)$ time.

Thus all operations run in:

$$\boxed{O(1)}$$

Exercise 11.1-4

★ Describe how to implement direct addressing on a huge array without initializing the entire array. All operations and initialization must run in $O(1)$ time.

Solution

Use two arrays:

- Huge array A
- Auxiliary stack S

Initially:

$$S = \emptyset$$

Each stored object contains:

- Key
- Pointer to its index in S

Each entry in S contains:

- Key
- Pointer back to A

INSERT(x):

- Push key onto stack S
- Store pointer in $A[x]$

SEARCH(x):

- Check $A[x]$
- Verify pointer consistency with S

DELETE(x):

- Remove element from stack S
- Fill hole with last stack element
- Update pointer in A

All operations take:

$$\boxed{O(1)}$$

Initialization also takes $O(1)$ since A is never explicitly initialized.

Hashing — Exercises 11.2

Collision Resolution by Chaining

Exercise 11.2-1

Suppose we use a hash function h to hash n distinct keys into an array T of length m . Assuming simple uniform hashing, what is the expected number of collisions? More precisely, what is the expected cardinality of

$$\{\{k, l\} : k \neq l \text{ and } h(k) = h(l)\}?$$

Solution

For each pair of keys k, l with $k \neq l$, define

$$X_{kl} = \begin{cases} 1 & \text{if } h(k) = h(l) \\ 0 & \text{otherwise} \end{cases}$$

Under simple uniform hashing:

$$\Pr(h(k) = h(l)) = \frac{1}{m}$$

Thus:

$$E[X_{kl}] = \frac{1}{m}$$

Let

$$Y = \sum_{k < l} X_{kl}$$

be the total number of collisions.

By linearity of expectation:

$$E[Y] = \sum_{k < l} E[X_{kl}] = \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2m}$$

$$\boxed{E[\text{collisions}] = \frac{n(n-1)}{2m}}$$

Exercise 11.2-2

Insert the keys 5, 28, 19, 15, 20, 33, 12, 17, 10 into a hash table with 9 slots using

$$h(k) = k \bmod 9.$$

Solution

Compute:

$$\begin{aligned}5 \bmod 9 &= 5 \\28 \bmod 9 &= 1 \\19 \bmod 9 &= 1 \\15 \bmod 9 &= 6 \\20 \bmod 9 &= 2 \\33 \bmod 9 &= 6 \\12 \bmod 9 &= 3 \\17 \bmod 9 &= 8 \\10 \bmod 9 &= 1\end{aligned}$$

Using insertion at head:

Slot	Chain
0	<i>NIL</i>
1	$10 \rightarrow 19 \rightarrow 28$
2	20
3	12
4	<i>NIL</i>
5	5
6	$33 \rightarrow 15$
7	<i>NIL</i>
8	17

Exercise 11.2-3

Professor Marley keeps each chain sorted. How does this affect running times?

Solution

Let average chain length be α .

- Successful search: $\Theta(1 + \alpha)$ (unchanged)
- Unsuccessful search: $\Theta(1 + \alpha)$ (may terminate early but same asymptotic)
- Insertion: $\Theta(1 + \alpha)$ (must find correct position)
- Deletion: $\Theta(1 + \alpha)$ if key given; $O(1)$ if pointer given

Sorting does not change asymptotic search time, but increases insertion cost.

Exercise 11.2-4

Suggest how to allocate/deallocate storage using a free list. Does it need to be doubly linked?

Solution

Each slot stores:

- A flag (free/used)
- Either (element + pointer)
- Or two pointers (for free list)

Free slots are linked in a free list.

Operations:

Insertion:

- Remove a slot from free list
- Place element

Deletion:

- Remove element
- Return slot to free list

All operations take expected $O(1)$ time.

The free list should be **doubly linked** if arbitrary deletions are needed. Otherwise, singly linked suffices if removing only from head.

Exercise 11.2-5

If $|U| > nm$, show that U contains n keys that hash to the same slot.

Solution

There are m slots.

Suppose each of $m - 1$ slots contains at most $n - 1$ elements.

Then remaining slot contains at least:

$$|U| - (m - 1)(n - 1)$$

Since $|U| > nm$:

$$> nm - (m - 1)(n - 1) = n + m - 1 \geq n$$

Thus one slot contains at least n keys.

Worst-case search time:

$$\boxed{\Theta(n)}$$

Exercise 11.2-6

Select a key uniformly at random in expected time

$$O(L(1 + 1/\alpha))$$

where L is longest chain.

Solution

View table as m rows, each row length $\leq L$.

Pick a random slot and a random position.

Probability of selecting a real key:

$$\frac{n}{mL}$$

Expected trials:

$$\frac{mL}{n} = \frac{L}{\alpha}$$

Each trial costs $O(1)$. Final chain scan costs $O(L)$.

Total expected time:

$$\boxed{O(L(1 + 1/\alpha))}$$

Open Addressing — Exercises 11.4

Exercise 11.4-1

Insert the keys

10, 22, 31, 4, 15, 28, 17, 88, 59

into a hash table of size $m = 11$ using:

- Linear probing
- Quadratic probing with $c_1 = 1$ and $c_2 = 3$
- Double hashing with

$$h_1(k) = k \bmod 11, \quad h_2(k) = 1 + (k \bmod 10)$$

Show the probe sequence for every key and give the final hash table for each technique.

Solution

For all three methods, the auxiliary hash function is

$$h'(k) = k \bmod 11.$$

The keys are inserted in the given order.

1. Linear Probing

The probe function is

$$h(k, i) = (h'(k) + i) \bmod 11, \quad i = 0, 1, 2, \dots$$

If the first position is occupied, we move to the next slot, wrapping around modulo 11.

Key	$h'(k)$	Probe sequence	Final slot
10	10	10	10
22	0	0	0
31	9	9	9
4	4	4	4
15	4	$4 \rightarrow 5$	5
28	6	6	6
17	6	$6 \rightarrow 7$	7
88	0	$0 \rightarrow 1$	1
59	4	$4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$	8

For example, key 59 starts at

$$59 \bmod 11 = 4.$$

Slots 4, 5, 6, 7 are already occupied, so it is placed in slot 8.

Final table — Linear Probing

Slot	0	1	2	3	4	5	6	7	8	9	10
Key	22	88	–	–	4	15	28	17	59	31	10

2. Quadratic Probing

With $c_1 = 1$ and $c_2 = 3$,

$$h(k, i) = (h'(k) + i + 3i^2) \bmod 11.$$

Key	$h'(k)$	Probe sequence	Final slot
10	10	10	10
22	0	0	0
31	9	9	9
4	4	4	4
15	4	$4 \rightarrow 8$	8
28	6	6	6
17	6	$6 \rightarrow 10 \rightarrow 9 \rightarrow 3$	3
88	0	$0 \rightarrow 4 \rightarrow 3 \rightarrow 8 \rightarrow 8 \rightarrow 3 \rightarrow 4 \rightarrow 0 \rightarrow 2$	2
59	4	$4 \rightarrow 8 \rightarrow 7$	7

A few nontrivial insertions are:

$$\begin{aligned} h(17, 0) &= (6 + 0 + 0) \bmod 11 = 6, \\ h(17, 1) &= (6 + 1 + 3) \bmod 11 = 10, \\ h(17, 2) &= (6 + 2 + 12) \bmod 11 = 9, \\ h(17, 3) &= (6 + 3 + 27) \bmod 11 = 3, \end{aligned}$$

so 17 is placed in slot 3.

For 59,

$$\begin{aligned} h(59, 0) &= 4, \\ h(59, 1) &= (4 + 1 + 3) \bmod 11 = 8, \\ h(59, 2) &= (4 + 2 + 12) \bmod 11 = 7, \end{aligned}$$

and slot 7 is empty, so 59 is inserted there.

Final table — Quadratic Probing

Slot	0	1	2	3	4	5	6	7	8	9	10
Key	22	–	88	17	4	–	28	59	15	31	10

Important: with the stated constants $c_1 = 1$, $c_2 = 3$, and $m = 11$, key 59 *does not fail*; it is inserted into slot 7.

3. Double Hashing

The probe function is

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod 11,$$

where

$$h_1(k) = k \bmod 11, \quad h_2(k) = 1 + (k \bmod 10).$$

Because 11 is prime and every $h_2(k)$ lies in $\{1, 2, \dots, 10\}$, every step size is relatively prime to 11. Thus a probe sequence can visit all table positions if necessary.

Key	$h_1(k)$	$h_2(k)$	Probe sequence	Final slot
10	10	1	10	10
22	0	3	0	0
31	9	2	9	9
4	4	5	4	4
15	4	6	$4 \rightarrow 10 \rightarrow 5$	5
28	6	9	6	6
17	6	8	$6 \rightarrow 3$	3
88	0	9	$0 \rightarrow 9 \rightarrow 7$	7
59	4	10	$4 \rightarrow 3 \rightarrow 2$	2

For example, for key 15,

$$h_1(15) = 4, \quad h_2(15) = 1 + (15 \bmod 10) = 6.$$

Hence

$$\begin{aligned} h(15, 0) &= (4 + 0 \cdot 6) \bmod 11 = 4, \\ h(15, 1) &= (4 + 1 \cdot 6) \bmod 11 = 10, \\ h(15, 2) &= (4 + 2 \cdot 6) \bmod 11 = 5. \end{aligned}$$

Slots 4 and 10 are occupied, so 15 is placed in slot 5.

For key 59,

$$h_1(59) = 4, \quad h_2(59) = 1 + (59 \bmod 10) = 10,$$

so the probes are

$$4 \rightarrow 3 \rightarrow 2,$$

and 59 is placed in slot 2.

Final table — Double Hashing

Slot	0	1	2	3	4	5	6	7	8	9	10
Key	22	–	59	17	4	15	28	88	–	31	10

Exercise 11.4-2

Write pseudocode for HASH-DELETE and modify HASH-INSERT to handle DELETED.

Solution

Algorithm 1 HASH-DELETE(T, k)

```
 $i \leftarrow 0$ 
repeat
   $j \leftarrow h(k, i)$ 
  if  $T[j] = k$  then
     $T[j] \leftarrow \text{DELETED}$ 
    return  $j$ 
  end if
   $i \leftarrow i + 1$ 
until  $T[j] = \text{NIL}$  or  $i = m$ 
error
```

Algorithm 2 HASH-INSERT(T, k)

```
 $i \leftarrow 0$ 
repeat
   $j \leftarrow h(k, i)$ 
  if  $T[j] = \text{NIL}$  or  $T[j] = \text{DELETED}$  then
     $T[j] \leftarrow k$ 
    return  $j$ 
  end if
   $i \leftarrow i + 1$ 
until  $i = m$ 
error
```

Exercise 11.4-3

Give expected probe bounds when $\alpha = 3/4$ and $\alpha = 7/8$.

Solution

Using Theorem 11.6:

$$E[\text{unsuccessful}] = \frac{1}{1 - \alpha}$$

Using Theorem 11.8:

$$E[\text{successful}] = \frac{1}{\alpha} \ln \frac{1}{1 - \alpha}$$

$$\alpha = 3/4$$

Unsuccessful:

$$\frac{1}{1 - 3/4} = 4$$

Successful:

$$\frac{1}{3/4} \ln 4 \approx 1.848$$

$$\alpha = 7/8$$

Unsuccessful:

$$\frac{1}{1 - 7/8} = 8$$

Successful:

$$\frac{1}{7/8} \ln 8 \approx 2.377$$

Exercise 11.4-4

★ Show that if $d = \gcd(m, h_2(k)) \geq 1$, then probe sequence examines only m/d slots.

Solution

Let:

$$h(k, i) = (h_1(k) + ih_2(k)) \bmod m$$

Let $d = \gcd(m, h_2(k))$.

Then $h_2(k)$ generates a cyclic subgroup modulo m of size:

$$\frac{m}{d}.$$

Thus the probe sequence repeats after m/d steps.

If $d = 1$, the sequence has full period m .

If $d > 1$, only m/d slots are examined.

Exercise 11.4-5

★ Find α such that unsuccessful search equals twice successful search.

Solution

Solve:

$$\frac{1}{1-\alpha} = 2 \cdot \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$$

Numerical solution:

$$\boxed{\alpha \approx 0.71533}$$